

Programming Project 2

Solving N-Queens Problem Using Hill-Climbing Search and Variants

Language: Python | By: Prithvi Koka and Krishna Patel

Introduction

The N-Queens Problem consists of placing N queens on an NxN chessboard so that no two queens can capture each other. That is, no two queens are allowed to be placed on the same row, the same column or the same diagonal. This project solves the problem with local search. Four algorithms were implemented, with the number of queens n supplied by the user at run time:

- Hill climbing search (no sideways moves)
- Hill-climbing search with sideways move
- Random-restart hill-climbing without sideways move
- Random-restart hill-climbing with sideways move

Experiments were run on the 8-queens problem ($n = 8$), as the assignment requires. This report describes the problem formulation, the program structure, the global variables, the heuristic function, and the remaining functions, and then presents the experimental results.

Problem Formulation

The problem is formulated as a ‘complete state’ local search problem: every state already has all the ‘n’ queens on the board, and the search tries to reduce the number of conflicts/overlaps.

Two queens in columns i and j ($i < j$) at rows `state[i]` and `state[j]` attack each other if `state[i] = state[j]` (same row) or $|state[i] - state[j]| = j - i$ (same diagonal). Because queens are in different columns by design, those two tests are sufficient.

Components:

1. **State:** A list state of ‘n’ integers. `State[c]` is the row ($0 - (n-1)$) of the queen in column c. It stores only one queen per column which rules out the column conflicts by construction and reduces the search space to n^n states.
2. **Initial State:** Each column receives a queen in a uniformly formed random row -> “function `configureRandomly`” in code.
3. **Actions/Successors:** Move one queen to a different row within its own column. Each state will then have $n(n-1)$ neighbors

4. **Heuristic h:** The number of pairs of queens that attack each other (in the same row, or column, or diagonal). The lower the 'h', the better, maximum is $n(n-1)/2$
5. **Goal Test:** $h(\text{state})=0$; no pair of queens attacks.
6. **Step Cost:** Each move costs 1. The "number of steps" reported in the results is the number of moves actually made, including sideways moves.

Program Structure

The whole program is in one python file: "Hill [Climbing.py](#)". To run the program, execute "python Hill Climbing.py" and enter n when prompted.

It has five layers:

1. **Board and state utilities:** Create, fill, copy, print, and convert between the state list and the 2-D board.
2. **Heuristic and neighbourhood:** 'calculateObjective' evaluates h, 'getNeighbor' scans all successors and returns the best ones.
3. **Search algorithms:** 'hillClimbing' (with an optional sideways-move flag) and 'randomRestartHillClimbing'.
4. **Experiment drivers:** 'runHillClimbingExperiment' and 'runRandomRestartExperiment' repeat the algorithms and collect statistics.
5. **Reporting and user input:** 'printExperimentTable', 'printSequences', 'readN' and 'main'.

Flow of main():

1. Obtain n from the command line (python nqueens.py 8) or, if no argument is given, prompt the user with readN(). Values below 4 are rejected, since $n = 2$ and $n = 3$ have no solution ($n = 1$ is trivial).
2. Print one demonstration solution found with random-restart hill climbing with sideways moves.
3. Experiment A: run plain hill climbing 50, 100, 200, 500, 1000 and 1500 times and print the results table.
4. Experiment B: repeat with sideways moves enabled (limit of 100).
5. Experiment C: run random-restart hill climbing for 500 trials, with and without sideways moves, and print the averages.
6. Print the search sequences for four random initial configurations, first without and then with sideways moves.

Global Variables

Variable	Default	Purpose
N	8	Board size / number of queens. Overwritten in main() with the value entered by the user
MAX_SIDEWAYS	100	Maximum number of consecutive sideways moves allowed before a run is declared a failure.
RUN_COUNTS	[50, 100, 200, 500, 1000, 1500]	Number of repetitions for each row of the hill-climbing result tables.
RESTART_TRIALS	500	Number of independent random-restart searches averaged in experiment C.
NUM_SEQUENCES	4	Number of random initial configurations whose search sequences are printed.

Objective/Heuristic Function

Function Code:

```
def calculateObjective(board, state):
    attacking = 0
    for i in range(N):
        for j in range(i + 1, N):
            if state[i] == state[j] or abs(state[i] - state[j]) == j - i:
                attacking += 1
    return attacking
```

‘calculateObjective(board, state)’ function returns h (the number of attacking pairs). It examines every pair of columns (i, j) with $i < j$ and adds 1 whenever the two queens share a row or a diagonal.

Each attacking pair is counted once, so $h = 0$ exactly when the state is a solution. The function runs in $O(n^2)$ time; evaluating all $n(n-1)$ neighbours of a state therefore costs $O(n^4)$, which dominates the running time for larger n . (Small note that the board argument is not needed by the computation; it is kept so that all functions share the same signature.)

Other Functions

Function	Description
configureRandomly(board, state)	Places one queen per column at a random row; fills state and marks the board.

Function	Description
printBoard(board)	Prints the board, 1 for a queen and 0 for an empty square.
printState(state)	Prints the state list (row index of each column).
compareStates(s1, s2)	Returns True if two states are identical.
fill(board, value)	Sets every square of the board to value.
generateBoard(board, state)	Rebuilds the 2-D board from a state list.
copyState(s1, s2)	Copies the contents of s2 into s1.
newBoard()	Returns a fresh $n \times n$ board of zeros.
getNeighbor(board, state)	Tries every (column, new row) move, evaluates h, and returns the lowest h found together with the list of all moves that achieve it.
hillClimbing(allowSideways, maxSideways, initialState, record)	Core search. Returns a dictionary with success, steps, final state, final h, board and (optionally) the recorded sequence.
randomRestartHillClimbing(allowSideways, maxSideways, maxRestarts)	Calls hillClimbing from fresh random states until one succeeds; returns restarts used and total steps over all attempts.
average(values)	Mean of a list, 0.0 for an empty list.
runHillClimbingExperiment(runs, allowSideways)	Runs hill climbing runs times; reports success/failure counts and rates and the average steps of successful and failed runs.
runRandomRestartExperiment(trials, allowSideways)	Runs random-restart search trials times; reports average restarts and average total steps.
printExperimentTable(title, rows)	Prints a formatted results table.
printSequences(title, allowSideways, count)	Runs hill climbing from count random starts and prints every state and h value along the path.
readN()	Prompts for n until an integer ≥ 4 is entered.
main()	Reads n, runs experiments A, B and C and prints the sequences.

Algorithm Details

Hill Climbing Search

This is steepest-ascent (here, steepest-descent on h) hill climbing. Starting from a random state, the algorithm repeatedly (1) stops if $h = 0$; (2) uses `getNeighbor` to find the best neighbour value; (3) stops with failure if that value is worse than, or equal to, the current h ; otherwise (4) moves to one of the best neighbours, chosen at random among ties with `random.choice`, and counts one step. A run fails when it is stuck at a local minimum or on a plateau.

Hill Climbing with Sideways Moves

When the best neighbour has the same h as the current state, the algorithm may move sideways instead of stopping, in the hope that the plateau is a shoulder rather than a flat local minimum. A counter `sidewaysUsed` limits this to `MAX_SIDEWAYS = 100` consecutive sideways moves; it is reset to zero whenever the search finds a strictly better state. Sideways moves count as steps. A run fails if h gets worse (strict local minimum) or the limit is exhausted on a plateau.

Random-restart Hill Climbing

Hill climbing is repeated from new random states until a solution is found. The program counts restarts as the number of failed attempts before the successful one (0 means the first attempt succeeded) and counts steps as the total number of moves over all attempts, failed ones included. This is done both without and with sideways moves.

Experimental Results with $n=8$

All results below come from a single execution of the program with $n = 8$. Because the starting states and tie-breaks are random, the figures vary slightly from run to run.

Hill-climbing search

Runs	Successes	Failures	Success %	Failure %	Avg steps (success)	Avg steps (failure)
50	5	45	10.00%	90.00%	4.00	3.04
100	14	86	14.00%	86.00%	4.29	3.08
200	36	164	18.00%	82.00%	4.19	3.20
500	67	433	13.40%	86.60%	4.18	3.04

Runs	Successes	Failures	Success %	Failure %	Avg steps (success)	Avg steps (failure)
1000	157	843	15.70%	84.30%	4.03	3.09
1500	205	1295	13.67%	86.33%	4.06	3.08

With 1500 runs, plain hill climbing solved 13.67% of the instances and failed on 86.33%. A successful run needed about 4.06 steps on average and a failing run stopped after about 3.08 steps, so the search gets stuck quickly.

The search sequences from four random initial configurations are shown below. Each row gives the state and its h value after every step. h falls on every step until the search either reaches h = 0 or gets stuck.

Sequence 1: failure after 3 steps

Step	State (row of each column)	h
0	[2, 4, 6, 5, 2, 7, 7, 1]	8
1	[2, 4, 6, 0, 2, 7, 7, 1]	4
2	[2, 4, 6, 0, 2, 5, 7, 1]	2
3	[2, 4, 6, 0, 3, 5, 7, 1]	1

Final board when the search stopped (1 = queen):

```

0 0 0 1 0 0 0 0
0 0 0 0 0 0 0 1
1 0 0 0 0 0 0 0
0 0 0 0 1 0 0 0
0 1 0 0 0 0 0 0
0 0 0 0 0 1 0 0
0 0 1 0 0 0 0 0
0 0 0 0 0 0 1 0

```

Sequence 2: success after 4 steps

Step	State (row of each column)	h
0	[6, 5, 2, 3, 4, 3, 1, 4]	9
1	[6, 5, 2, 2, 4, 3, 1, 4]	5
2	[6, 5, 2, 2, 5, 3, 1, 4]	3
3	[6, 0, 2, 2, 5, 3, 1, 4]	2
4	[6, 0, 2, 7, 5, 3, 1, 4]	0

Final board (1 = queen):

```

0 1 0 0 0 0 0 0
0 0 0 0 0 0 1 0
0 0 1 0 0 0 0 0
0 0 0 0 0 1 0 0
0 0 0 0 0 0 0 1
0 0 0 0 1 0 0 0
1 0 0 0 0 0 0 0
0 0 0 1 0 0 0 0

```

Sequence 3: failure after 4 steps

Step	State (row of each column)	h
0	[3, 7, 3, 4, 3, 4, 5, 6]	12
1	[3, 7, 3, 4, 2, 4, 5, 6]	6
2	[3, 7, 3, 4, 2, 1, 5, 6]	4
3	[3, 7, 2, 4, 2, 1, 5, 6]	3
4	[3, 7, 2, 4, 1, 1, 5, 6]	2

Final board when the search stopped (1 = queen):

```

0 0 0 0 0 0 0 0
0 0 0 0 1 1 0 0
0 0 1 0 0 0 0 0
1 0 0 0 0 0 0 0
0 0 0 1 0 0 0 0
0 0 0 0 0 0 1 0
0 0 0 0 0 0 0 1
0 1 0 0 0 0 0 0

```

Sequence 4: failure after 4 steps

Step	State (row of each column)	h
0	[3, 3, 0, 0, 6, 0, 3, 6]	10
1	[3, 3, 0, 4, 6, 0, 3, 6]	6
2	[3, 7, 0, 4, 6, 0, 3, 6]	3
3	[3, 7, 0, 4, 6, 1, 3, 6]	2
4	[5, 7, 0, 4, 6, 1, 3, 6]	1

Final board when the search stopped (1 = queen):

```

0 0 1 0 0 0 0 0
0 0 0 0 0 1 0 0
0 0 0 0 0 0 0 0
0 0 0 0 0 0 1 0
0 0 0 1 0 0 0 0
1 0 0 0 0 0 0 0
0 0 0 0 1 0 0 1
0 1 0 0 0 0 0 0

```

Hill-climbing search with sideways moves (limit = 100)

Runs	Successes	Failures	Success %	Failure %	Avg steps (success)	Avg steps (failure)
50	44	6	88.00%	12.00%	19.02	21.17
100	95	5	95.00%	5.00%	19.42	64.40
200	189	11	94.50%	5.50%	18.49	67.91
500	472	28	94.40%	5.60%	19.06	63.21
1000	925	75	92.50%	7.50%	19.08	64.96
1500	1417	83	94.47%	5.53%	19.21	66.01

With 1500 runs, allowing sideways moves raised the success rate to 94.47% (failure rate 5.53%). The price is longer runs: 19.21 steps on average for successes and 66.01 for failures. The failure average is a mixture of runs that hit a strict local minimum early and runs that used up the 100-move plateau allowance.

The search sequences from four random initial configurations are shown below. Here h can stay constant for several consecutive steps (sideways moves across a plateau) before dropping again.

Sequence 1: success after 20 steps

Step	State (row of each column)	h
0	[5, 0, 6, 6, 0, 1, 3, 6]	7
1	[5, 0, 6, 4, 0, 1, 3, 6]	4
2	[5, 0, 6, 4, 0, 1, 3, 7]	2
3	[5, 0, 6, 4, 6, 1, 3, 7]	1
4	[5, 0, 0, 4, 6, 1, 3, 7]	1
5	[5, 2, 0, 4, 6, 1, 3, 7]	1
6	[5, 7, 0, 4, 6, 1, 3, 7]	1

Step	State (row of each column)	h
7	[5, 2, 0, 4, 6, 1, 3, 7]	1
8	[5, 0, 0, 4, 6, 1, 3, 7]	1
9	[5, 2, 0, 4, 6, 1, 3, 7]	1
10	[5, 0, 0, 4, 6, 1, 3, 7]	1
11	[5, 0, 6, 4, 6, 1, 3, 7]	1
12	[5, 0, 0, 4, 6, 1, 3, 7]	1
13	[5, 2, 0, 4, 6, 1, 3, 7]	1
14	[5, 7, 0, 4, 6, 1, 3, 7]	1
15	[5, 0, 0, 4, 6, 1, 3, 7]	1
16	[5, 7, 0, 4, 6, 1, 3, 7]	1
17	[5, 7, 0, 4, 6, 1, 3, 2]	1
18	[5, 7, 0, 4, 6, 1, 6, 2]	1
19	[5, 7, 0, 4, 7, 1, 6, 2]	1
20	[5, 3, 0, 4, 7, 1, 6, 2]	0

Final board (1 = queen):

```

0 0 1 0 0 0 0 0
0 0 0 0 0 1 0 0
0 0 0 0 0 0 0 1
0 1 0 0 0 0 0 0
0 0 0 1 0 0 0 0
1 0 0 0 0 0 0 0
0 0 0 0 0 0 1 0
0 0 0 0 1 0 0 0

```

Sequence 2: success after 6 steps

Step	State (row of each column)	h
0	[0, 4, 4, 7, 2, 1, 4, 5]	10
1	[0, 4, 4, 7, 2, 1, 3, 5]	5
2	[0, 4, 4, 7, 3, 1, 3, 5]	3
3	[0, 4, 6, 7, 3, 1, 3, 5]	2
4	[2, 4, 6, 7, 3, 1, 3, 5]	2
5	[2, 4, 6, 7, 3, 1, 7, 5]	2

Step	State (row of each column)	h
6	[2, 4, 6, 0, 3, 1, 7, 5]	0

Final board (1 = queen):

```

0 0 0 1 0 0 0 0
0 0 0 0 0 1 0 0
1 0 0 0 0 0 0 0
0 0 0 0 1 0 0 0
0 1 0 0 0 0 0 0
0 0 0 0 0 0 0 1
0 0 1 0 0 0 0 0
0 0 0 0 0 0 1 0

```

Sequence 3: success after 21 steps

Step	State (row of each column)	h
0	[7, 0, 0, 3, 6, 7, 4, 7]	8
1	[7, 0, 0, 3, 6, 7, 4, 1]	5
2	[7, 0, 0, 3, 5, 7, 4, 1]	3
3	[7, 0, 6, 3, 5, 7, 4, 1]	2
4	[7, 0, 6, 3, 5, 7, 4, 2]	2
5	[7, 0, 6, 3, 1, 7, 4, 2]	1
6	[3, 0, 6, 3, 1, 7, 4, 2]	1
7	[7, 0, 6, 3, 1, 7, 4, 2]	1
8	[7, 0, 6, 3, 1, 6, 4, 2]	1
9	[7, 0, 5, 3, 1, 6, 4, 2]	1
10	[2, 0, 5, 3, 1, 6, 4, 2]	1
11	[7, 0, 5, 3, 1, 6, 4, 2]	1
12	[4, 0, 5, 3, 1, 6, 4, 2]	1
13	[7, 0, 5, 3, 1, 6, 4, 2]	1
14	[2, 0, 5, 3, 1, 6, 4, 2]	1
15	[2, 0, 5, 3, 1, 6, 4, 1]	1
16	[2, 0, 5, 3, 0, 6, 4, 1]	1
17	[2, 0, 7, 3, 0, 6, 4, 1]	1
18	[2, 4, 7, 3, 0, 6, 4, 1]	1

Step	State (row of each column)	h
19	[2, 4, 7, 3, 0, 6, 5, 1]	1
20	[2, 4, 7, 3, 0, 6, 1, 1]	1
21	[2, 4, 7, 3, 0, 6, 1, 5]	0

Final board (1 = queen):

```

0 0 0 0 1 0 0 0
0 0 0 0 0 0 1 0
1 0 0 0 0 0 0 0
0 0 0 1 0 0 0 0
0 1 0 0 0 0 0 0
0 0 0 0 0 0 0 1
0 0 0 0 0 1 0 0
0 0 1 0 0 0 0 0

```

Sequence 4: success after 19 steps

Step	State (row of each column)	h
0	[1, 3, 0, 1, 6, 7, 5, 7]	7
1	[1, 4, 0, 1, 6, 7, 5, 7]	4
2	[1, 4, 0, 1, 6, 2, 5, 7]	2
3	[1, 4, 0, 3, 6, 2, 5, 7]	1
4	[1, 4, 0, 3, 6, 2, 5, 1]	1
5	[1, 4, 0, 3, 6, 2, 5, 2]	1
6	[1, 4, 0, 3, 6, 7, 5, 2]	1
7	[1, 4, 0, 3, 6, 2, 5, 2]	1
8	[1, 4, 0, 3, 6, 2, 5, 1]	1
9	[1, 4, 7, 3, 6, 2, 5, 1]	1
10	[1, 4, 7, 3, 6, 2, 5, 0]	1
11	[1, 4, 7, 3, 6, 2, 5, 1]	1
12	[1, 4, 0, 3, 6, 2, 5, 1]	1
13	[4, 4, 0, 3, 6, 2, 5, 1]	1
14	[4, 4, 7, 3, 6, 2, 5, 1]	1
15	[4, 0, 7, 3, 6, 2, 5, 1]	1
16	[4, 4, 7, 3, 6, 2, 5, 1]	1

Step	State (row of each column)	h
17	[4, 0, 7, 3, 6, 2, 5, 1]	1
18	[4, 2, 7, 3, 6, 2, 5, 1]	1
19	[4, 2, 7, 3, 6, 0, 5, 1]	0

Final board (1 = queen):

```

0 0 0 0 0 1 0 0
0 0 0 0 0 0 0 1
0 1 0 0 0 0 0 0
0 0 0 1 0 0 0 0
1 0 0 0 0 0 0 0
0 0 0 0 0 0 1 0
0 0 0 0 1 0 0 0
0 0 1 0 0 0 0 0

```

Random-restart hill-climbing search (500 trials each)

Variant	Average restarts	Average steps
Without sideways moves	6.38	23.40
With sideways moves	0.05	22.68

Without sideways moves, about 6.38 restarts (i.e. roughly 7.4 attempts) were needed on average, consistent with the 14% single-run success rate in experiment A ($1 / 0.14 = 7$ attempts). With sideways moves, a restart was almost never needed (0.05 on average). Interestingly, the total number of steps is almost the same for both variants (23.40 versus 22.68): the variant without sideways moves makes many short failed attempts, while the variant with sideways moves makes fewer but longer ones.

Demonstration solution

The program also prints one solution for the chosen n. For this execution it was found after 0 restart(s) and 6 steps, state [3, 6, 2, 7, 1, 4, 0, 5]:

```

0 0 0 0 0 0 1 0
0 0 0 0 1 0 0 0
0 0 1 0 0 0 0 0
1 0 0 0 0 0 0 0
0 0 0 0 0 1 0 0
0 0 0 0 0 0 0 1
0 1 0 0 0 0 0 0
0 0 0 1 0 0 0 0

```

Discussion and Conclusion

Method	Success rate	Avg steps (success)	Avg steps (failure)
Hill climbing	13.67%	4.06	3.08
Hill climbing + sideways	94.47%	19.21	66.01

- Plain hill climbing is fast but unreliable: it solves only about one instance in seven because the 8-queens landscape has many local minima and plateaus.
- Sideways moves let the search cross plateaus and raise the success rate to roughly 94%, at the cost of about five times as many steps per success.
- Random restarts make the search complete in practice (probability of success approaches 1). They need about 6-7 restarts without sideways moves and almost none with them, while using about 23 steps in total either way.